

DATE : 23.09.2026
DT/NT : DT
LESSON : LINUX
SUBJECT: WORKING WITH FILES & SEARCH
SESSION : 3
BATCH : B 418/DE014

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com

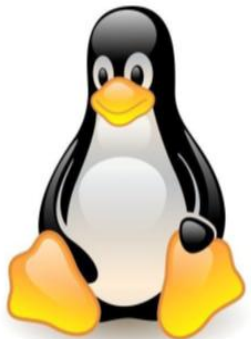


+1 (585) 304 29 59



Table of Contents

- ▶ CLI Text Editors
- ▶ Reading & Following Files
- ▶ Finding Files
- ▶ Searching Text



CLI Text Editors



Vi / Vim Editor



Why learn Vim?

- Common on Linux servers and minimal environments
- Works completely in the terminal
- Fast for configuration and script edits
- Powerful once you understand its modes

Key idea

Vim is modal. The same key can behave differently depending on the current mode.

For Day 3, we focus on only the essential modes and commands.

Server skill

CLI only

Config files

Vim: The Essential Modes



NORMAL MODE

Navigate, delete, copy, paste, search.

Press ``Esc`` to return here.

INSERT MODE

Type and edit text.

Press ``i`` to enter Insert mode.

COMMAND-LINE MODE

Save, quit, and run editor commands.

From Normal mode, press ``:``.

Mental model: `Esc` → Normal | `i` → Insert | `:` → Command-line

Vim: Essential Commands



OPEN / SAVE / EXIT

```
vim app.conf
```

i	# insert mode
Esc	# normal mode
:w	# save
:q	# quit
:wq	# save and quit
:q!	# quit without saving

USEFUL NORMAL-MODE COMMANDS

/error	# search forward
n	# next match
dd	# delete current line
yy	# copy current line
p	# paste
u	# undo
Ctrl+r	# redo

Nano Editor



Why Nano?

- Easy for beginners
- Shortcut hints are visible on screen
- Good for quick edits
- No modal editing model

OPEN A FILE

```
nano app.conf
```

```
# Edit the file directly
```

Important: do not assume Nano is installed on every minimal server image. Check with:

command `-v nano`

Nano: Essential Shortcuts

SHORTCUTS

Ctrl+O	Write/save file
Ctrl+X	Exit
Ctrl+W	Search
Ctrl+K	Cut current line
Ctrl+U	Paste / uncut
Alt+U	Undo
Alt+E	Redo

Shortcut notation

Nano often displays `^` for Ctrl.

Example: `^X` means Ctrl+X.

Which editor should I use?

Use whichever editor is available and comfortable. For remote Linux administration, be able to perform basic edits in Vim even if Nano is your preference.



Reading & Following Files



head — Read the Beginning of a File



``head`` prints the first 10 lines by default.

EXAMPLES

```
head app.log  
head -n 5 app.log  
head -n 20 /etc/passwd
```

DevOps use

Quickly inspect the beginning of configuration files, exported data, logs, or generated artifacts without opening an editor.

tail — Read the End of a File



``tail`` prints the last 10 lines by default.

EXAMPLES

```
tail app.log  
tail -n 5 app.log  
tail -n 50 app.log
```

Why DevOps uses it

The newest events in a traditional log file are usually at the end, so ``tail`` is a fast first inspection tool.

tail -f — Follow a Growing Log



FOLLOW MODE

```
tail -f app.log
```

```
# Stop with Ctrl+C
```

What happens?

`tail -f` keeps waiting for new data and prints new lines as the file grows.

Class demo

Terminal 1: `tail -f app.log`

Terminal 2: `echo "INFO deployment completed" >> app.log`

Watch the new line appear immediately.

cat — Display or Combine Small Files



EXAMPLES

```
cat app.conf  
cat file1 file2  
cat file1 file2 > combined.txt
```

Use it wisely

`cat` is great for short files. For long files, use `less` so you can navigate and search instead of flooding the terminal.

less — Navigate Large Files



BASIC NAVIGATION

```
less app.log  
less /etc/passwd
```

q	quit
/word	search forward
n	next match
G	end of file
g	beginning

Why prefer `less`?

It gives interactive navigation and search without editing the file. This makes it useful for configuration files, logs, and command output.

less with Command Output



You can also send command output into `less` with a pipe.

EXAMPLES

```
ps aux | less  
find . -type f | less  
cat app.log | less
```

Note

`cat file | less` works, but `less file` is simpler when reading a single file.

more and tac — Useful, but Secondary



more

A simple pager that displays output screen by screen.

For interactive navigation, ``less`` is generally more capable.

tac

GNU ``tac`` prints file lines in reverse order.

Example:

```
`tac app.log`
```

Useful occasionally, but not a core DevOps command.

Choose the Right File-Reading Command



Quick whole file

`cat config.txt`

Best for short files.

Beginning / end

`head -n 20 file`

`tail -n 50 file`

Fast inspection.

Large / live files

`less app.log`

`tail -f app.log`

Navigate or follow.

Finding Files & Searching Text



find — Search the Filesystem

``find`` walks a directory tree and evaluates files against each entry.

BASIC PATTERN

```
find <starting-point> <file-name>
```

```
find . -name "app.conf"
```

```
find ~/linux-day3 -type f -name "*.log"
```

Quote wildcard patterns such as ``"*.log"`` so the shell does not expand them before ``find`` sees them.

find by Name and Type

EXAMPLES

```
find . -name "app.conf"
```

```
find . -iname "readme.md"
```

```
find . -type f
```

```
find . -type d
```

```
find . -type f -name "*.log"
```

-name vs -iname

`-name` is case-sensitive.

`-iname` is case-insensitive.

-type

`-type f` → regular files

`-type d` → directories



find by Time and Size

PRACTICAL EXAMPLES

```
# modified in the last 24 hours  
find . -type f -mtime -1
```

```
# older than 7 days  
find . -type f -mtime +7
```

```
# larger than 10 MiB  
find . -type f -size +10M
```

Important precision

GNU `find` measures `-mtime` in rounded 24-hour periods.

For `-size`, `M` means MiB (1024×1024 bytes).

Use these as operational filters, not as calendar-date wording.



grep — Search Text Inside Files



``grep`` prints lines that match a pattern.

BASIC SEARCH

```
grep "ERROR" app.log
grep -i "error" app.log
grep -w "ERROR" app.log
```

File search vs text search

``find`` searches filesystem entries such as names, types, time, and size.

``grep`` searches text content.

grep: Common Options

EXAMPLES

```
grep -i "error" app.log
grep -n "ERROR" app.log
grep -v "DEBUG" app.log
grep -c "ERROR" app.log
grep -w "ERROR" app.log
```

What the options mean

- `-i` ignore case
- `-n` show line numbers
- `-v` invert the match
- `-c` count matching lines
- `-w` match whole words



grep: Show Context Around a Match



CONTEXT OPTIONS

```
grep -A 3 "ERROR" app.log  
grep -B 2 "ERROR" app.log  
grep -C 2 "ERROR" app.log
```

Context

`-A N` N lines after
`-B N` N lines before
`-C N` N lines before and after

Very useful when the important log event spans several lines.

grep with Pipes — Inspect Command Output



EXAMPLES

```
ps aux | grep nginx  
history | grep ssh  
find . -type f | grep "\.log$"  
  
grep -i "error" app.log | less
```

DevOps pattern

A common workflow is:

command → pipe → grep → less

Use one command to produce data, another to filter it, and a pager when the output is long.

Mini-Lab: Investigate an Application Log



RUN THESE COMMANDS

```
mkdir -p ~/linux-day3
cd ~/linux-day3

printf "INFO start\nWARN disk\nERROR db\nINFO recovered" >> app.log

head -n 2 app.log
tail -n 2 app.log
grep -n "ERROR" app.log
grep -C 1 "ERROR" app.log
less app.log
```

Live-log extension

Terminal 1:

```
`tail -f app.log`
```

Terminal 2:

```
`echo "INFO recovered" >> app.log`
```

Goal

Students should be able to answer:

- Where is the error?
- What happened before/after it?
- How do I watch for new events?

Day 3 Challenge



- Create `~/day3-challenge/{config,logs,docs}`.
- Create/edit a configuration file with Vim or Nano.
- Create a 10+ line log and inspect its beginning and end.
- Use `tail -f` while another terminal appends a new event.
- Find all `.log` files under the challenge directory.
- Search the log for `ERROR` and show line numbers plus two lines of context.
- Open the final log in `less` and search inside it.



Do you have any questions?

Day 3 takeaway:

Edit files → inspect files → follow logs → find files → search text.

These are everyday Linux troubleshooting skills for AWS & DevOps.